Skill – Week9

Session – 17

1. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

```
  GNU nano 8.7.1                                jobs.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX 10

struct Job {
    int id;
    pid_t pid;
    char command[50];
    char status[20];
};

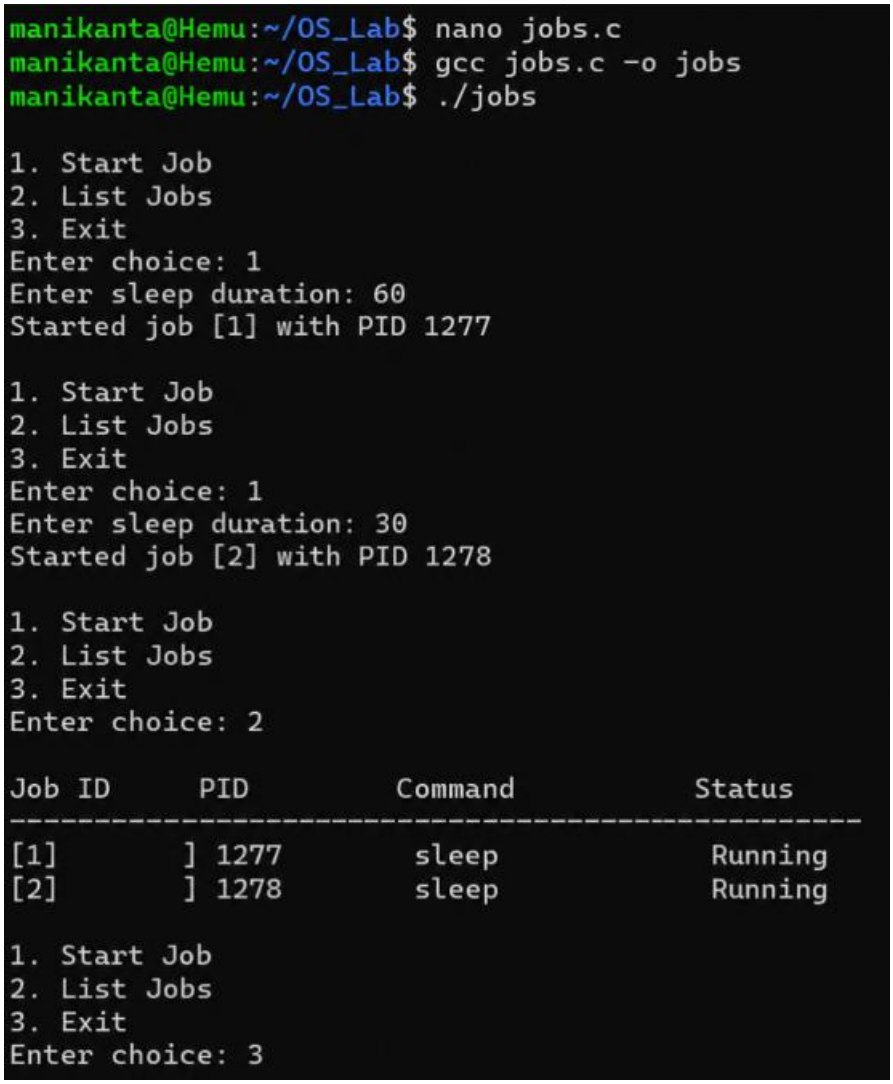
struct Job jobs[MAX];
int count = 0;

// Create a background job
void create_job(char *cmd, int seconds) {
    pid_t pid = fork();

    if (pid == 0) {
        char time[10];
        sprintf(time, "%d", seconds);
        execlp("sleep", "sleep", time, NULL);
        perror("execlp");
        exit(1);
    }

    if (pid > 0) {
        jobs[count].id = count + 1;
        jobs[count].pid = pid;

        snprintf(jobs[count].command, 50, "%s", cmd);
        snprintf(jobs[count].status, 20, "Running");
```



Report: The program used fork(), waitpid(), and printf() to create jobs, update their states, and display job IDs and status information.

2. To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

```
  GNU nano 8.7.1                    job_control.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>
#include <string.h>

#define MAX 10

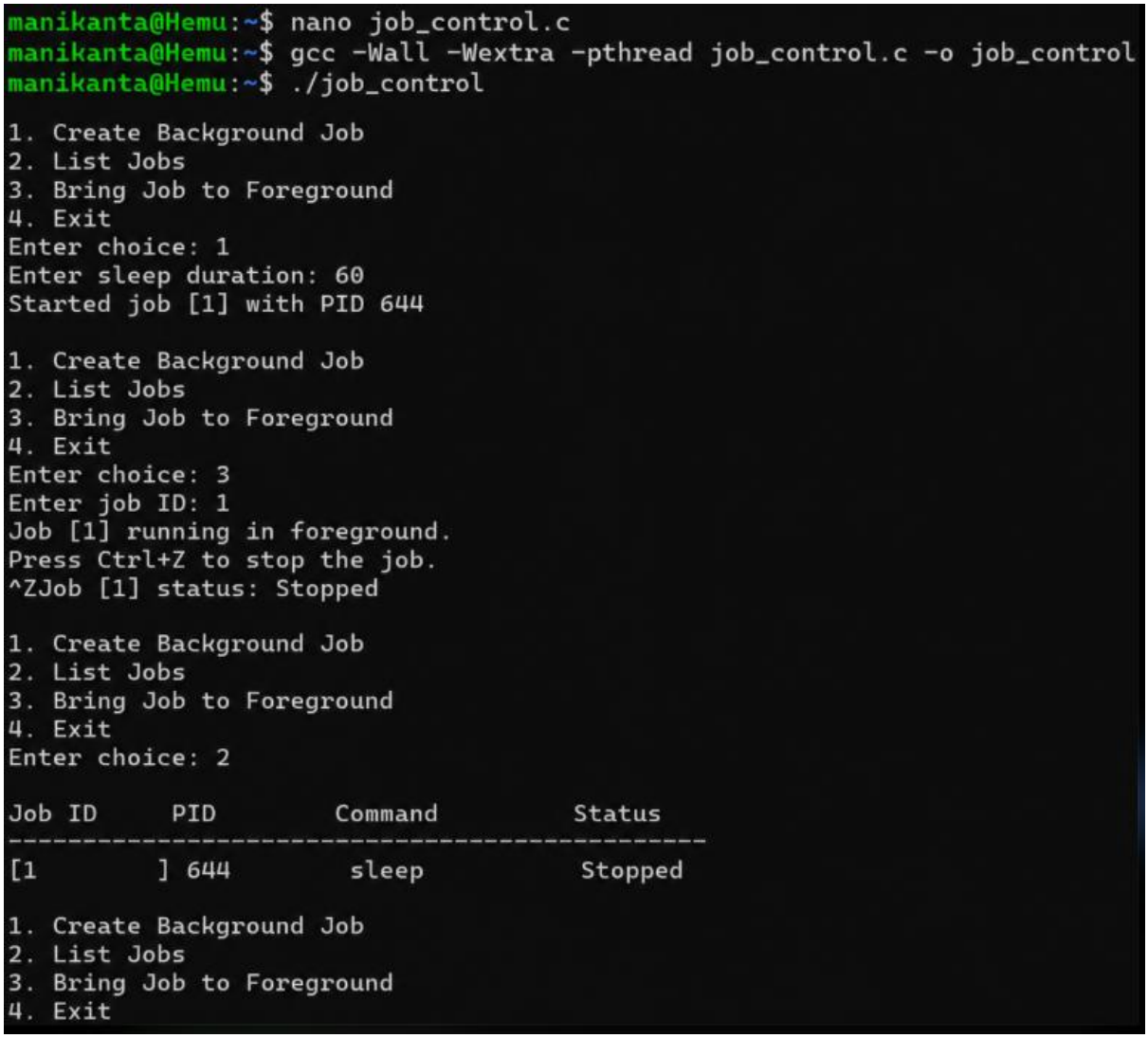
struct Job {
    int id;
    pid_t pid;
    char status[20];
};

struct Job jobs[MAX];
int count = 0;

pid_t shell_pgid;
int terminal_fd = STDIN_FILENO;

void update_jobs() {
    for (int i = 0; i < count; i++) {
        int status;
        pid_t result = waitpid(jobs[i].pid, &status,
                               WNOHANG | WUNTRACED | WCONTINUED);

        if (result > 0) {
            if (WIFEXITED(status) || WIFSIGNALED(status))
                strcpy(jobs[i].status, "Done");
            else if (WIFSTOPPED(status))
                strcpy(jobs[i].status, "Stopped");
            else if (WIFCONTINUED(status))
                strcpy(jobs[i].status, "Running");
        }
    }
```



Report: The program used tcsetpgrp(), SIGCONT, and waitpid() to switch foreground jobs, resume processes, and update job states.

Session 18:

1. To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

```
  GNU nano 8.7.1                          signal_demo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>

volatile sig_atomic_t int_count = 0;
volatile sig_atomic_t term_count = 0;

void handle_sigint(int sig)
{
    int_count++;
    write(STDOUT_FILENO, "\nSIGINT received (Ctrl+C)\n", 26);
}

void handle_sigterm(int sig)
{
    term_count++;
    write(STDOUT_FILENO, "\nSIGTERM received\n", 18);
}

int main()
{
    pid_t pid;
    sigset_t mask, oldmask;
    struct sigaction sa;

    sa.sa_handler = handle_sigint;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGINT, &sa, NULL);

    sa.sa_handler = handle_sigterm;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGTERM, &sa, NULL);
```

```
manikanta@Hemu:~$ nano signal_demo.c
manikanta@Hemu:~$ gcc -Wall -Wextra signal_demo.c -o signal_demo
signal_demo.c: In function 'handle_sigint':
signal_demo.c:11:24: warning: unused parameter 'sig' [-Wunused-parameter]
   11 | void handle_sigint(int sig)
      |                    ~~~~^~~
signal_demo.c: In function 'handle_sigterm':
signal_demo.c:17:25: warning: unused parameter 'sig' [-Wunused-parameter]
   17 | void handle_sigterm(int sig)
      |                     ~~~~^~~
manikanta@Hemu:~$ ./signal_demo
Parent PID: 1202
Parent Process Group: 1202

Child created with PID: 1203

Blocking SIGINT for 10 seconds...
SIGINT blocked: 10 seconds remaining

Child PID: 1203
Child Process Group: 1202
Child running...
SIGINT blocked: 9 seconds remaining
Child running...
SIGINT blocked: 8 seconds remaining
Child running...
SIGINT blocked: 7 seconds remaining
Child running...
SIGINT blocked: 6 seconds remaining
Child running...
SIGINT blocked: 5 seconds remaining
Child running...
^C
SIGINT received (Ctrl+C)
Child running...
SIGINT blocked: 4 seconds remaining
Child running...
SIGINT blocked: 3 seconds remaining
Child running...
SIGINT blocked: 2 seconds remaining
Child running...
```

```
Unblocking SIGINT...

SIGINT received (Ctrl+C)
SIGINT is now unblocked.
Press Ctrl+C to test signal delivery.
Press Ctrl+Z to suspend the foreground process group.
^Z
[1]+  Stopped                 ./signal_demo
manikanta@Hemu:~$ fg
./signal_demo
Child running...
Child running...
Child running...
^Z
[1]+  Stopped                 ./signal_demo
```

Report: SIGINT blocked in the parent does not automatically block it in the child forever; the child inherits the signal mask at the time of fork(), and the parent later unblocks its

own mask. Ctrl+C and Ctrl+Z are generated by the terminal for its foreground process group.

2. To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery

```
  GNU nano 8.7.1                          job_resume.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_JOBS 10

struct Job
{
    pid_t pid;
    int status;
};

struct Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid)
{
    if (job_count < MAX_JOBS)
    {
        jobs[job_count].pid = pid;
        jobs[job_count].status = 0;
        job_count++;
    }
}

void update_status()
{
    int status;
    pid_t pid;

    for (int i = 0; i < job_count; i++)
    {
        pid = waitpid(jobs[i].pid, &status,
                      WNOHANG | WUNTRACED | WCONTINUED);
```

```
manikanta@Hemu:~$ nano job_resume.c
manikanta@Hemu:~$ gcc -Wall -Wextra job_resume.c -o job_resume
manikanta@Hemu:~$ ./job_resume

1. Create Background Job
2. Stop Job
3. Resume Job
4. Show Job Table
5. Exit
Enter choice: 1
Background job created. PID = 1277

1. Create Background Job
2. Stop Job
3. Resume Job
4. Show Job Table
5. Exit
Enter choice: Child 1277 running: 1
Child 1277 running: 2
Child 1277 running: 3
4

------ JOB TABLE ------
Job 1 | PID: 1277 | Status: Running

1. Create Background Job
2. Stop Job
3. Resume Job
4. Show Job Table
5. Exit
Enter choice: Child 1277 running: 4
Child 1277 running: 5
Child 1277 running: 6
Child 1277 running: 7
Child 1277 running: 8
Child 1277 running: 9
2Child 1277 running: 10
Child 1277 running: 11
```

```
------ JOB TABLE ------
Job 1 | PID: 1277 | Status: Running
Enter job number to stop: Child 1277 running: 12
Child 1277 running: 13
Child 1277 running: 14
Child 1277 running: 15
Child 1277 running: 16
^Z
[2]+  Stopped                 ./job_resume
```

Report: The program successfully demonstrates process suspension and recovery using SIGSTOP and SIGCONT, while waitpid() monitors process state changes and the job table maintains the current status of background jobs.